<YOUR COLLEGE / DEPARTMENT NAME>
(write your college and department here)

CSA09 – PROGRAMMING IN JAVA

ASSIGNMENT 2 – CO2 REPORT

Strings, Collections Framework, hashCode–equals, Generics, Comparable / Comparator

Item	Details
Student name	<YOUR NAME>
Register number	<REGISTER NUMBER>
Course	CSA09 – Programming in Java
Assignment / CO	Assignment 2 – CO2 (Sessions 15–26), Individual, 50 marks
Java version used	OpenJDK 21 (java -version)
GitHub repository	https://github.com/<your-username>/CSA09-<RegisterNumber>-Assignment2
Submission date	<DATE>

Note on the repository: the repository contains one folder per question (Q1 … Q7) with the Java source, the CSV files holding the raw measurements, the console output and the output screenshots. The README.md in the repository root explains how to compile and run every program.

Contents and Experimental Method

Contents

Section	Topic	Marks
Q1	String, StringBuilder or StringBuffer? (railway e-ticket)	6
Q2	When is ArrayList an appropriate choice? (music playlist)	4
Q3	The hashCode–equals contract (fest attendance)	6
Q4	Choosing a Map (hospital pharmacy inventory)	6
Q5	Generic ranking component (sports leaderboard)	7
Q6	Priority, order and fairness in a queue (emergency triage)	8
Q7	Day-4 design challenge: diagnose, redesign, validate	13
App.	Repository structure, how to run, list of raw data files	–

How I measured

The whole assignment is about **Predict → Design → Implement → Measure → Analyse → Evaluate**, so every question follows the same pattern: I first wrote a prediction with the reason, then wrote the program, then measured, and then compared the prediction with the observed result.

- **Timing:** System.nanoTime() around the operation being studied. Every experiment is repeated (normally 5 times) and the **average** is reported. Before the measured runs I run a **warm-up** so that the JIT compiler has already compiled the code; without a warm-up the first numbers are much slower and misleading.
- **Memory:** I used the number of bytes allocated by the current thread (com.sun.management.ThreadMXBean.getThreadAllocatedBytes). This is more reliable than "used memory before and after" because the garbage collector can run in between. For Q1 I also print the used memory before and after, as the question asks.
- **Operation counts:** where possible I also counted operations (for example how many patients were examined in Q7). Operation counts do not change from run to run, so they are the most reliable evidence.
- **Raw data:** every program writes its numbers to a .csv file in its own folder. The tables and graphs in this report are generated from those files (scripts/make_graphs.py).
- **Honest limitation:** timings below about 1–2 ms (for example the redesign in Q7 for small n) are affected by garbage collection and JIT noise, and they change slightly every time the program is run. I therefore base my conclusions on trends over several sizes and on operation counts, not on a single value.

Test machine: the programs were executed with OpenJDK 21 on Linux. (Write your own laptop details here: processor, RAM, operating system – your numbers will differ from mine, but the trends must be the same.)

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer?

Problem context: a railway system builds an e-ticket by joining many fields (PNR, names, coach, berth, fare, journey details) into one text block. The current code uses the + operator inside a loop. I investigated whether the choice of String, StringBuilder or StringBuffer really matters.

1.1 Prediction before implementation

Reasoning. A String is immutable. When I write ticket = ticket + field + "|" in a loop, Java cannot change the old String, so it creates a **new String object** and copies all the characters built so far plus the new field. If a ticket has n fields of average length L, the copying for field number i costs about i·L characters, so the total copying is L·(1 + 2 + … + n) = L·n(n+1)/2, which is **$O(n^2)$**. A StringBuilder keeps one internal character array that is expanded by doubling, so each append copies only the new field (amortised O(1)) and the whole ticket costs **O(n)**. StringBuffer does the same work but every method is **synchronized**, so I expect it to be a little slower than StringBuilder when only one thread uses it.

Workload	My prediction
W1 – 1,000 short tickets, 10 fields each	Small work. The three approaches should be almost equal (difference negligible); String + may be a little slower.
W2 – 10,000 tickets, 50 fields each	String + clearly slower (a few times) and it creates about 50 times more objects; StringBuilder faster; StringBuffer about 5–25% slower than StringBuilder.
W3 – 20 tickets, 5,000 fields each (group booking / long log)	The quadratic effect dominates: String + should be hundreds of times slower and allocate gigabytes; StringBuilder and StringBuffer almost equal.

1.2 Design of the workloads

I designed three deliberately different workloads: W1 (short tickets, many repetitions so that the time can be measured), W2 (the “normal” case suggested in the question: 10,000 tickets with 50 fields) and W3 (my own workload: very long tickets, such as a group booking or a combined booking log, 5,000 fields each). In addition, a **scaling experiment** builds one ticket with n = 500, 1000, … 32,000 fields so that the growth rate can be seen directly.

1.3 Implementation

The program TicketBuilder.java contains one method for each approach. The two main ones are shown below; String.join() and StringJoiner are included for the alternative-concept part.

```
static String buildPlus(String[] fields) {
    String ticket = "";
    for (int i = 0; i < fields.length; i++) {
        ticket = ticket + fields[i] + "|";      // new String object every time
    }
    return ticket;
}

static String buildStringBuilder(String[] fields) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < fields.length; i++) {
        sb.append(fields[i]).append("|");
    }
    return sb.toString();
}
```

1.4 Measured results

Average of 5 runs after warm-up. “Alloc” is the total memory allocated while building all tickets once; “Used before / after” is the heap in use before and after one run (affected by the garbage collector, so the allocation column is the better memory measure). “Est. Strings” is my estimate of the String objects created (tickets × fields for the + operator; about 2 per ticket for the others).

Workload	Approach	Time (ms)	Alloc (MB)	Used MB before → after	Est. Strings
W1 1000×10	String +	0.295	1.1	1.6 → 2.7	10,000
W1 1000×10	StringBuilder	0.249	0.5	1.6 → 2.2	2,000
W1 1000×10	StringBuffer	0.281	0.5	1.6 → 2.2	2,000
W1 1000×10	String.join	0.176	0.2	1.5 → 1.9	2,000
W1 1000×10	StringJoiner	2.010	0.3	1.5 → 1.9	2,000
W2 10000×50	String +	20.124	206.8	1.5 → 3.6	500,000
W2 10000×50	StringBuilder	6.779	31.0	1.5 → 15.8	20,000
W2 10000×50	StringBuffer	7.646	31.1	1.5 → 15.9	20,000
W2 10000×50	String.join	3.725	9.9	1.5 → 11.7	20,000
W2 10000×50	StringJoiner	7.330	13.4	1.5 → 15.2	20,000
W3 20×5000	String +	363.503	4175.3	1.9 → 16.4	100,000
W3 20×5000	StringBuilder	1.332	7.3	1.9 → 9.3	40
W3 20×5000	StringBuffer	1.631	7.3	1.9 → 9.2	40
W3 20×5000	String.join	0.776	2.1	1.9 → 3.9	40
W3 20×5000	StringJoiner	2.486	2.9	1.9 → 4.8	40

Table 1.1 – Q1 workloads (from Q1/q1_workloads.csv)

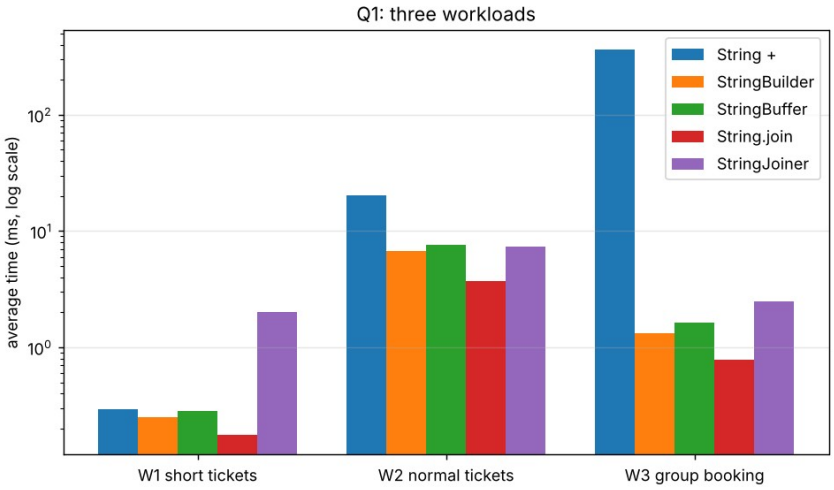


Figure 1.1 – Time for the three workloads (logarithmic scale)

Scaling experiment: time (ms) to build **one** ticket with n fields.

n (fields)	String +	StringBuilder	StringBuffer	String.join	StringJoiner
500	0.183	0.008	0.008	0.004	0.012
1,000	1.669	0.010	0.012	0.008	0.021
2,000	4.215	0.026	0.024	0.013	0.045
4,000	12.581	0.042	0.052	0.030	0.088
8,000	48.870	0.088	0.100	0.061	0.177
16,000	189.302	0.224	0.159	0.078	0.217

n (fields)	String +	StringBuilder	StringBuffer	String.join	StringJoiner
32,000	795.838	0.707	0.662	0.370	0.885

Table 1.2 – Scaling with the number of fields (from Q1/q1_scaling.csv)

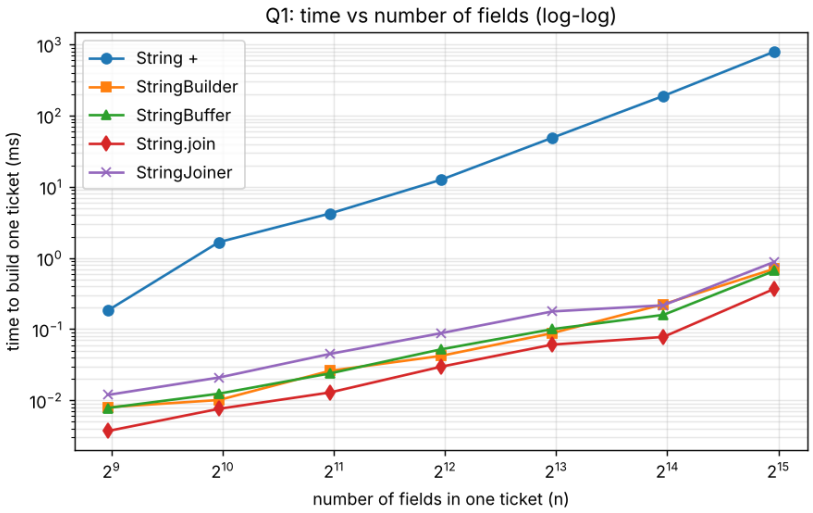


Figure 1.2 – Log–log plot: String + rises with slope about 2, the others with slope about 1

Growth rate. When n doubled from 8,000 to 16,000, the String + time grew by a factor of 3.9, and from 16,000 to 32,000 by 4.2. A factor of about 4 for doubling n is the signature of $O(n^2)$. For StringBuilder the factor was 2.5 and 3.2, close to 2 (O(n)); the small timings are noisy, which explains why it is not exactly 2.

1.5 Prediction compared with observation

- **W1 (short tickets):** String + took 0.295 ms against 0.249 ms for StringBuilder, so the difference is negligible, as predicted. In an earlier run of the same program String + was about 4–5 times slower in W1 because the JIT had not yet optimised it, and one StringJoiner value jumped to 2 ms because of a garbage-collection pause. This showed me that very short workloads are noisy; the absolute difference is only about one millisecond per 1,000 tickets.
- **W2:** String + took 20.1 ms against 6.8 ms for StringBuilder (3.0 times slower) and allocated 206.8 MB against 31.0 MB (6.7 times more). The prediction "a few times slower" was correct.
- **W3:** String + took 363.5 ms against 1.33 ms (273 times slower) and allocated about 4,175 MB, which is roughly 570 times more than StringBuilder. My prediction of "hundreds of times" was correct; the allocation is the reason the garbage collector is also kept busy.
- **StringBuffer vs StringBuilder:** StringBuffer was slower in W2 (7.6 ms vs 6.8 ms) and W3 (1.63 ms vs 1.33 ms). The cost of synchronisation is real but small when there is no contention (about 10–25%). It allocates the same memory.
- **What surprised me:** String.join() was the fastest of all. It first calculates the total length and then builds the result in one exactly-sized array, so there is no growth copying. Also, memory "used after" looked larger for StringBuilder than for String +, but that only shows how much garbage had not been collected yet; the allocation column is the true comparison.

1.6 Design decision: multithreaded booking server

Per-ticket text → StringBuilder. Each thread builds its own ticket in a local variable, so no other thread can see the object. Synchronisation is then wasted work, and StringBuilder is the fastest of the mutable classes.

Shared booking log → StringBuffer (or, better, one lock around whole entries). I tested the shared log with 4 threads each appending 50,000 lines to the same object, repeated 20 times. The StringBuilder lost characters in **3 of 20 trials** (for example 18, 12, 18 characters missing) and threw an exception in 3 of them, because two threads updated its internal count and array at the same time. The StringBuffer gave the correct length in **20 of 20 trials**. The race is not seen in every run, which is exactly why it is dangerous: the code "works" in testing and fails in production.

A limitation I must mention: StringBuffer makes each single append() call safe, but a log entry made of several appends can still be mixed with another thread's entry. To keep a whole line together I would build the line in a local StringBuilder first and then append it with a single append call, or use a synchronized block / a ConcurrentLinkedQueue for log lines.

1.7 Alternative concept and complexity analysis

If String.join(), StringJoiner or String.format() were used instead of StringBuilder, the solution would change as follows:

```
String ticket1 = String.join("|", fields);           // one call, fields already in a
String[] / List

StringJoiner sj = new StringJoiner("|", "[", "]");    // delimiter, prefix, suffix
for (String f : fields) sj.add(f);
String ticket2 = sj.toString();

String ticket3 = String.format("%s|%s|%s|%s", pnr, name, coach, berth);   // fixed
number of fields only
```

Approach	Time for n fields	Extra space (garbage) created	Remark
String + in loop	$O(n^2)$ characters copied ($L \cdot n(n+1)/2$)	$O(n^2)$ characters allocated in total, n String objects; only O(n) alive at one time	Slow and creates much garbage for large n
StringBuilder	O(n) amortised	O(n): one builder plus one result; growth copies add at most about 2× the final length	Best general choice
StringBuffer	O(n) (+ lock cost per call)	Same as StringBuilder	Use only when shared by threads
String.join()	O(n)	O(n): one result array, almost no garbage	Fastest in my tests; needs all fields ready
StringJoiner	O(n)	O(n): keeps the pieces then joins them	Convenient for delimiter / prefix / suffix
String.format()	O(n) but a large constant (parses the format string every call)	O(n) plus a Formatter object	Good for readability with few fields; slowest per call

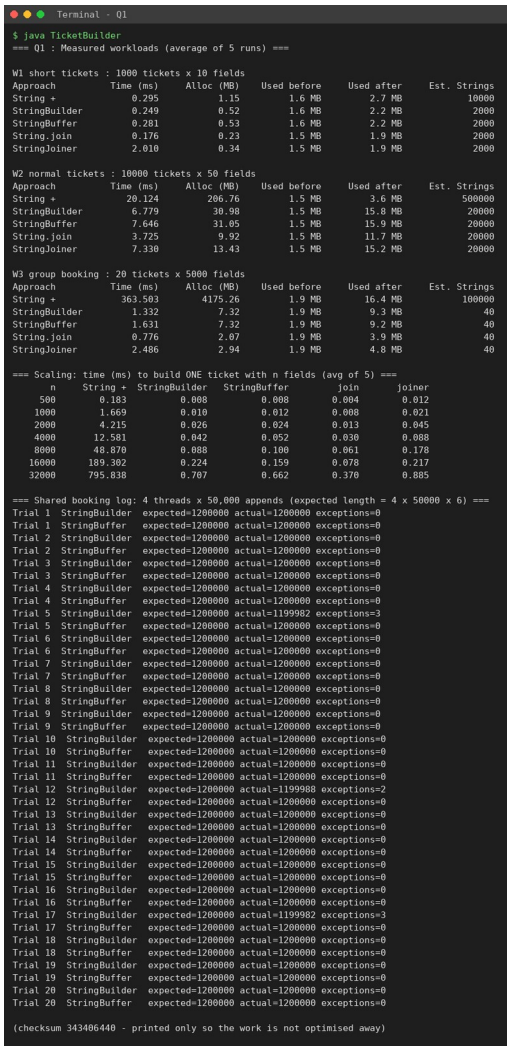
Table 1.3 – Complexity comparison

Derivation. With repeated +, step i copies the i fields already built, so the characters copied are $L \cdot (1 + 2 + \ldots + n) = L \cdot n(n+1)/2 = O(n^2)$. With StringBuilder, each append copies only the new field, and the occasional array growth copies the old contents; because the capacity doubles, those copies sum to $1 + 2 + 4 + \ldots \leq 2 \cdot$(final length), so the total is O(n). (From Java 9 the compiler turns a single expression such as a + b + c into one efficient concatenation, but a + inside a loop still creates a new String for every iteration, which is what the measurements show.)

1.8 Conclusion for Q1

The choice matters when the text is long or built many times, and it hardly matters for a few short strings. For the e-ticket generator I recommend StringBuilder (or String.join when the fields are already in an array), StringBuffer only for an object shared between threads, and never + inside a loop. The evidence: 273-times slower and 570-times more allocation for + in W3, correct log length in all trials only for StringBuffer.

Screenshot of the program output:



```
Terminal - Q1

$ java TicketBuilder
=== Q1 : Measured workloads (average of 5 runs) ===

W1 short tickets : 1000 tickets x 10 fields
Approach          Time (ms)     Alloc (MB)    Used before     Used after     Est. Strings
String +              0.295           1.15         1.6 MB         2.7 MB            10000
StringBuilder         0.249           0.52         1.6 MB         2.2 MB             2000
StringBuffer          0.281           0.53         1.6 MB         2.2 MB             2000
String.join           0.176           0.23         1.5 MB         1.9 MB             2000
StringJoiner          2.010           0.34         1.5 MB         1.9 MB             2000

W2 normal tickets : 10000 tickets x 50 fields
Approach          Time (ms)     Alloc (MB)    Used before     Used after     Est. Strings
String +             20.124         206.76         1.5 MB         3.6 MB           500000
StringBuilder         6.779          30.98         1.5 MB        15.8 MB            20000
StringBuffer          7.646          31.05         1.5 MB        15.9 MB            20000
String.join           3.725           9.92         1.5 MB        11.7 MB            20000
StringJoiner          7.330          13.43         1.5 MB        15.2 MB            20000

W3 group booking : 20 tickets x 5000 fields
Approach          Time (ms)     Alloc (MB)    Used before     Used after     Est. Strings
String +            363.503        4175.26         1.9 MB        16.4 MB           100000
StringBuilder         1.332           7.32         1.9 MB         9.3 MB               40
StringBuffer          1.631           7.32         1.9 MB         9.2 MB               40
String.join           0.776           2.07         1.9 MB         3.9 MB               40
StringJoiner          2.486           2.94         1.9 MB         4.8 MB               40

=== Scaling: time (ms) to build ONE ticket with n fields (avg of 5) ===
      n     String +  StringBuilder   StringBuffer         join       joiner
    500        0.183          0.008          0.008        0.004        0.012
   1000        1.669          0.010          0.012        0.008        0.021
   2000        4.215          0.026          0.024        0.013        0.045
   4000       12.581          0.042          0.052        0.030        0.088
   8000       48.870          0.088          0.100        0.061        0.178
  16000      189.302          0.224          0.159        0.078        0.217
  32000      795.838          0.707          0.662        0.370        0.885

=== Shared booking log: 4 threads x 50,000 appends (expected length = 4 x 50000 x 6) ===
Trial 1  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 1  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 2  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 2  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 3  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 3  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 4  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 4  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 5  StringBuilder  expected=1200000 actual=1199982 exceptions=3
Trial 5  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 6  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 6  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 7  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 7  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 8  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 8  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 9  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 9  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 10  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 10  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 11  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 11  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 12  StringBuilder  expected=1200000 actual=1199988 exceptions=2
Trial 12  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 13  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 13  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 14  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 14  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 15  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 15  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 16  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 16  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 17  StringBuilder  expected=1200000 actual=1199982 exceptions=3
Trial 17  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 18  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 18  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 19  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 19  StringBuffer   expected=1200000 actual=1200000 exceptions=0
Trial 20  StringBuilder  expected=1200000 actual=1200000 exceptions=0
Trial 20  StringBuffer   expected=1200000 actual=1200000 exceptions=0

(checksum 343406440 - printed only so the work is not optimised away)
```

Figure 1.3 – Console output of TicketBuilder.java

Q2 – When Is ArrayList an Appropriate Engineering Choice?

Problem context: a music playlist editor adds songs at the end, inserts at chosen positions, removes songs and jumps to the k-th song. The app stores each playlist in an ArrayList<Song>.

2.1 Prediction

ArrayList stores the references in an array. Reading position k is one array access (O(1)). Inserting or removing at position p must move all elements after p (O(n − p)), so front is the worst place. Adding at the end is O(1) amortised. LinkedList is the opposite: a node can be inserted at the front in O(1), but reaching position k means following k links (O(n)). So I predicted: ArrayList wins get(k) by a huge margin; LinkedList wins insert/remove at the front; **in the middle both are O(n)**, but I expected the LinkedList to still lose because following pointers scattered in memory is slower than the array shift, which is done by a fast System.arraycopy.

2.2 Experiment design

- Playlist sizes N = 10,000, 50,000 and 200,000 songs (three sizes, as required).
- For each operation, 1,000 operations are timed (the whole-list iteration is timed once and divided by N). The experiment is repeated 5 times and averaged, after a warm-up.
- Operations: add at end, insert at front, insert in middle, remove at front, remove in middle, get k-th (1,000 random positions generated with a fixed seed so all lists see the same positions), iterate the whole list with for-each.
- ArrayDeque is included where it has the operation (it has no index-based operations, so those cells are n/a).

2.3 Results

Time in **microseconds per operation** (per element for iteration), average of 5 runs.

Operation	AL 10k	LL 10k	AL 50k	LL 50k	AL 200k	LL 200k	AD 200k
add at end	0.102	0.084	0.073	0.072	0.073	0.081	0.0096
insert at front	0.463	0.093	5.60	0.075	20.79	0.078	0.011
insert in middle	1.20	17.65	3.27	88.94	9.41	696	n/a
remove at front	0.971	0.948	4.84	0.046	18.68	0.080	0.0010
remove in middle	0.261	9.45	2.37	102	9.53	592	n/a
get k-th (random)	0.102	5.84	0.456	53.71	0.166	393	n/a
iterate whole list	0.0030	0.197	0.017	0.017	0.0032	0.028	0.0081

Table 2.1 – AL = ArrayList, LL = LinkedList, AD = ArrayDeque (from Q2/q2_list_results.csv)

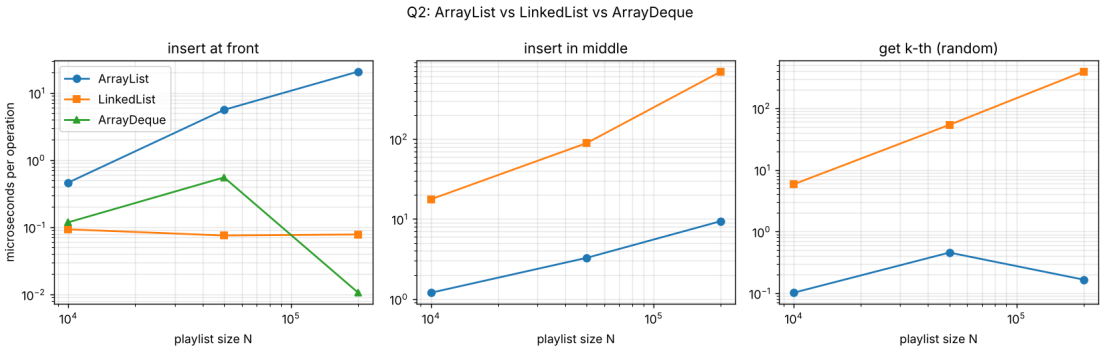


Figure 2.1 – The three operations where the structures differ most

Observations

- **get(k):** at N = 200,000 ArrayList needs 0.166 µs and LinkedList 393 µs, a difference of more than 1,000 times, and LinkedList grows with N exactly as O(n) predicts.
- **Insert at front:** ArrayList grows from 0.463 µs (N = 10k) to 20.79 µs (N = 200k), i.e. linearly, while LinkedList stays at about 0.078 µs. ArrayDeque is also constant (0.011 µs).
- **Insert / remove in the middle:** my prediction was right: both are O(n) but LinkedList is far worse (696 µs against 9.41 µs at 200k). The LinkedList has to walk to the middle first. So "LinkedList is good for insertion" is only true if you already hold an iterator at the position.
- **Add at end and iteration:** about the same for both (O(1) and O(n)); the LinkedList iteration is slightly slower per element because its nodes are scattered in memory (poor cache use). Small values such as 0.003 µs are close to the timer resolution.

2.4 Operating boundary for ArrayList

ArrayList is a reasonable choice when the playlist is mostly appended to, read by index and iterated, and front or middle changes are occasional. A user playlist usually has hundreds or a few thousand songs; at N = 10,000 even a front insert costs only about 0.5 µs, which nobody can notice.

I would stop recommending it when front or middle insertions/removals become a regular, high-volume part of the workload on large lists. The cost per operation is proportional to N (21 µs at 200,000 songs). A practical boundary I can defend from the data: if the system performs more than about 10,000 front/middle changes per second on lists of 100,000+ elements, the shifting alone would consume about 20% of one CPU core (10,000 × 21 µs = 0.21 s per second). Below that, ArrayList stays simpler and faster than LinkedList for everything else.

2.5 Changed condition: the "Up Next" queue

The Up Next queue adds songs at both ends and removes from the front. For an ArrayList both "add at front" and "remove from front" are O(n). I measured 100,000 rounds of addLast + addFirst + removeFirst on queues already holding N songs:

Songs in queue	ArrayList (ms)	LinkedList (ms)	ArrayDeque (ms)
1,000	908.1	28.9	8.3
10,000	1095.2	20.0	11.3
50,000	1769.4	29.2	14.5

Table 2.2 – Up Next queue, 100,000 rounds (from Q2/q2_upnext_results.csv)

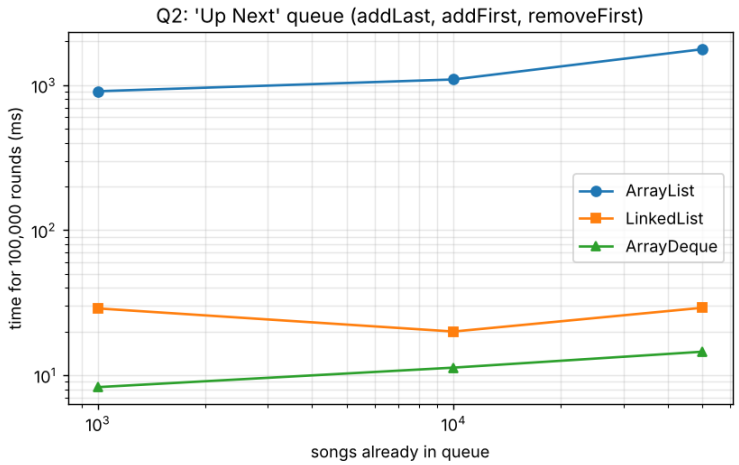


Figure 2.2 – Up Next queue

ArrayList is about 100–120 times slower than ArrayDeque for this pattern (908 vs 8 ms, 1,095 vs 11 ms, 1,769 vs 15 ms). **My re-evaluation:** the original recommendation (ArrayList for the main playlist) stays, but the Up Next queue should be an **ArrayDeque**. LinkedList is also fast (O(1) at both ends), but each element needs a separate node object (about 24 bytes plus the reference) and scattered memory, and it was about 2–3.5 times slower than ArrayDeque in my runs. The price of ArrayDeque is that it has no get(k) and no insert in the middle; for an Up Next queue that is acceptable because users only look at the front and reorder rarely.

2.6 Alternative concept and complexity analysis

Operation	ArrayList	LinkedList	ArrayDeque	Plain array, manual shifting
Add at end	O(1) amortised	O(1)	O(1) amortised	O(1) if space; O(n) to copy into a bigger array
Insert at front	O(n)	O(1)	O(1) amortised	O(n)
Insert in middle	O(n) (fast arraycopy)	O(n) (walk to position)	not supported	O(n) (element-by-element shift)
Get k-th	O(1)	O(n)	not supported	O(1)
Iterate all	O(n)	O(n)	O(n)	O(n)
Space per element	1 array slot (4–8 bytes) + spare capacity up to about 50%	node object (about 24 bytes) + its element reference	1 array slot + spare capacity (up to 100%)	1 array slot, fixed capacity chosen in advance

Table 2.3 – Complexity of the operations

Plain array with manual shifting. The solution would keep a Song[] and an int size, and shift elements with a loop for each insert:

```
for (int i = size; i > position; i--) arr[i] = arr[i - 1];   // shift right
arr[position] = newSong;
size++;
```

The complexity is the same as ArrayList (O(n) insert, O(1) get), but I also have to write the resizing code myself. I measured 1,000 inserts:

N	Operation	Plain array (ms)	ArrayList (ms)
10,000	insert at front	17.797	1.288
10,000	insert in middle	13.421	0.269
50,000	insert at front	51.889	4.831
50,000	insert in middle	22.107	4.103
200,000	insert at front	127.194	21.432
200,000	insert in middle	83.102	10.519

Table 2.4 – Plain array vs ArrayList (from Q2/q2_array_results.csv)

In five of the six rows the manual loop is 5 to 14 times slower than ArrayList (one row at N = 10,000 even about 50 times) although both are O(n), because ArrayList uses System.arraycopy, a native block move, while my loop moves one reference per iteration. This is a good reminder that Big-O hides the constant factor.

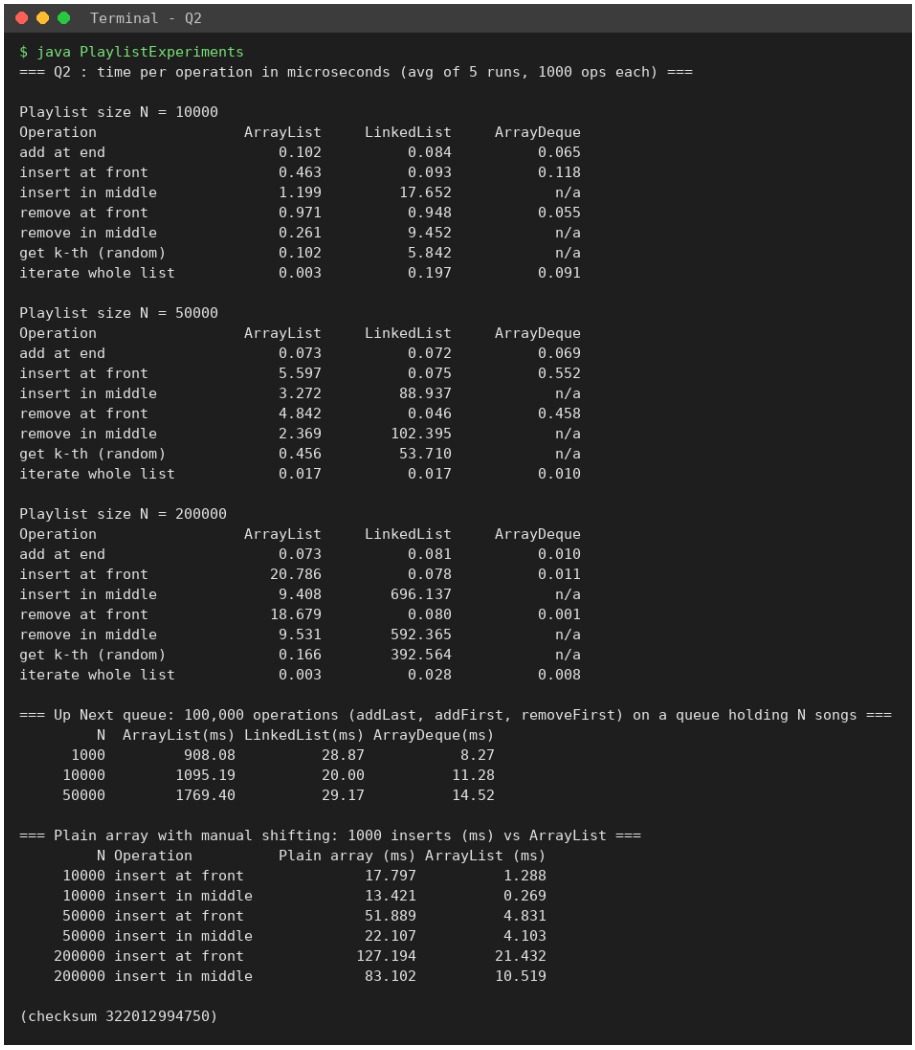


Figure 2.3 – Console output of PlaylistExperiments.java

Q3 – Engineering the hashCode–equals Contract

Problem context: student ID cards are scanned at several gates and stored in a HashSet<Student>. The junior developer's Student class overrides equals() (compares regNo) but does **not** override hashCode().

3.1 Tests, predictions and observed results

For each test I wrote the prediction first (it is printed by the program before the result, in HashContractExperiments.java).

Test	Scans	Prediction (size) and reason	Observed
1 – appears to work	3 different students, and the same card object scanned twice	Size 3. Different regNos are different students, and the repeated scan is the same object, so even the default identity hashCode is identical and equals() is true.	Bad class: 3, Good class: 3 – both correct
2 – exposes duplicates	Arun (new object), Divya, Arun again (another new object)	Bad class: 3 (a duplicate!). The two Arun objects are equal by equals() but have different identity hash codes, so they go to different buckets and equals() is never asked. Good class: 2.	Bad: 3, Good: 2. Bad set contents: [192311002 Divya, 192311001 Arun, 192311001 Arun]
2b – bigger	100 students × 10 scans = 1,000 scans	Bad: about 1,000 (nothing removed), Good: 100.	Bad: 1000, Good: 100
3a – mutate key	Add Meena, then change her regNo after insertion (GoodStudent, regNo not final)	contains() fails for the old and the new regNo, and adding the new regNo gives size 2: the object sits in the bucket of its old hash code.	contains(m) = false, contains(old) = false, contains(new) = false; after adding another object with the new regNo: size 2
3b – TreeSet	Two different students both named Ravi, TreeSet ordered by name only	Size 1: TreeSet uses compare(), not equals(), and compare returns 0 for equal names.	r1.equals(r2) = false but TreeSet size = 1

Table 3.1 – Test design, prediction and observation

For Test 2 I also printed the hash codes: the two equal students had hashCode 1229416514 (bucket 5) and 2016447921 (bucket 1) in the default HashMap table of size 16, so they were stored in different buckets. For the corrected class both objects give the same hash code and the same bucket. (The exact numbers change from run to run because the default hashCode is based on object identity; the fact that they differ is what matters.)

3.2 Inquiry

Why the duplicates appear

The contract says: **if a.equals(b) is true then a.hashCode() must equal b.hashCode()**. The junior's class breaks it, because Object.hashCode() returns a value tied to the object's identity. Two Student objects with the same regNo are equal but get different hash codes.

How HashSet uses hashCode() and equals()

HashSet is built on a HashMap. add(x) first computes hash = x.hashCode() (spread with h ^ (h >>> 16)) and the bucket index (table.length − 1) & hash. It then looks only inside that bucket: for every stored element it first compares the stored hash with the new hash and only if they are equal calls equals(). So equals() is never reached for objects in different buckets. If the bucket is empty or no element matches, the new element is added. That is why a correct hashCode is needed for correctness and also for speed (a good spread keeps buckets short, giving O(1) average).

What happens when a field used in hashCode() is changed after insertion

The element stays in the bucket calculated from the old hash code. After the change, contains() and remove() look in the bucket of the new hash code and do not find it (Test 3a). The object is "lost"

inside the set (a memory leak), and the same student can be added again, which gives a duplicate. The cure is to use **immutable fields** in equals/hashCode (regNo declared private final, as in the Complete-the-Code class) or never to change such fields while the object is in a hash-based collection.

Correcting the class and verifying the correction

hashCode() must use the same field(s) as equals(): only regNo (not name, because two scans may spell the name differently). I verified it in three ways: (1) the Attendance program now prints 2 instead of 3; (2) Test 2 and the 1,000-scan run give 2 and 100; (3) the program checks the contract rules – reflexive, symmetric, transitive, null returns false, and equal objects have equal hash codes – and all passed.

3.3 Complete the Code

First run **without** TODO 1 (AttendanceBefore.java):



Then TODO 1 completed:

```
// TODO 1 (completed): hashCode uses ONLY regNo, the same field used by equals()
@Override
public int hashCode() {
    return Objects.hash(regNo);
}
```



Version	Output	Reason
Before TODO 1	Unique students: 3	Arun's two objects have different identity hash codes → different buckets → both stored.
After TODO 1	Unique students: 2	Both Arun objects have the same hash code (from regNo) → same bucket → equals() returns true → second add is rejected. This matches the expected output.

3.4 Alternative concepts and complexity

TreeSet<Student> with a Comparator on regNo: no hashCode is needed, because the TreeSet (a red-black tree) decides duplicates with compare() == 0: `new TreeSet<>(Comparator.comparing(s -> s.regNo))`. Every scan costs O(log n) comparisons. **HashMap<String, Student> keyed by regNo:** `map.putIfAbsent(regNo, student)`; the key is a String whose hashCode/equals are already correct, so the junior's bug cannot happen, and I can store extra information (e.g. first gate). Cost O(1) average per scan.

Scans	Unique	HashSet (ms)	TreeSet (ms)	HashMap (ms)
10,000	5,000	1.38	3.07	3.09
100,000	50,000	12.28	44.72	13.41
1,000,000	500,000	183.42	341.37	91.77

Table 3.2 – Time to process n scans, half of them repeats (from Q3/q3_dedup_results.csv)

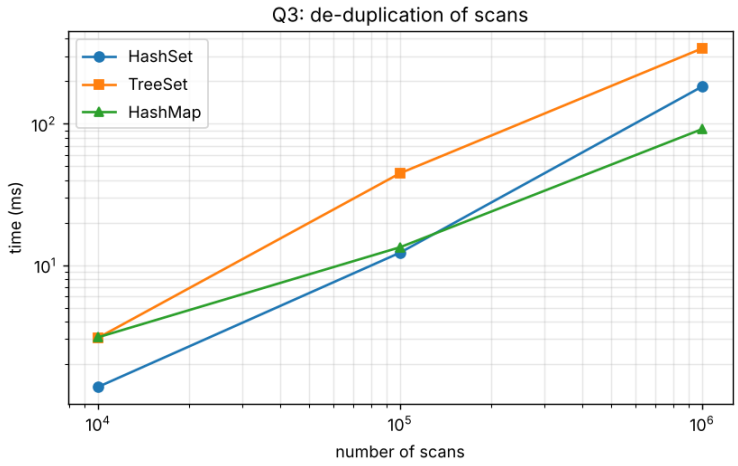


Figure 3.1 – De-duplication time

HashSet and HashMap are close to each other (HashSet is implemented with a HashMap inside, so both are O(1) average; the small differences at 1,000,000 scans come from resizing and garbage collection). TreeSet was about 2–3.6 times slower than HashSet, matching O(log n) against O(1). **Space** for n unique students: HashSet and HashMap both need a table of buckets plus one entry node per element (hash, key, value, next ≈ 32 bytes); a TreeSet needs one tree node per element (key, left, right, parent, colour ≈ 40 bytes) and no table. All are O(n). The advantage of TreeSet is that the students come out sorted by regNo, which a HashSet cannot give.

```
Terminal - Q3
$ java HashContractExperiments
=== Q3 : hashCode - equals contract experiments ===

TEST 1 : every student scanned once, one card object scanned twice (same object)
PREDICTION: size 3 for BadStudent and GoodStudent. Distinct students have different regNo, and the
            repeated scan is the SAME object, so even the default identity hashCode is equal.
OBSERVED  : BadStudent set size = 3, GoodStudent set size = 3

TEST 2 : the same student scanned again at another gate (a NEW object each time)
PREDICTION: BadStudent -> 3 (duplicate appears: equals says equal but hashCodes differ, so the
            second object goes to a different bucket and equals() is never called).
            GoodStudent -> 2.
OBSERVED  : BadStudent set size = 3, GoodStudent set size = 2
            Bad set contents : [192311002 Divya, 192311001 Arun, 192311001 Arun]
            x.equals(y) = true   x.hashCode() = 1229416514 (bucket 5)   y.hashCode() = 2016447921 (bucket 1)
            good: hashCode 822376295 and 822376295 -> same bucket 3
            Bigger run (100 students x 10 scans = 1000 scans): Bad size = 1000, Good size = 100  (correct answer is 100)

TEST 3a : change regNo AFTER the student is stored in the HashSet (GoodStudent, regNo not final)
PREDICTION: contains(...) fails for both old and new regNo, and adding the new regNo gives size 2,
            because the object sits in the bucket of the OLD hash but now has the NEW regNo.
OBSERVED  : before change  size=1 contains(m)=true
            after change   contains(m)=false  contains(new 192311010)=false  contains(new 192311099)=false
            after adding another object with regNo 192311099: size=2 (same student twice!)

TEST 3b : TreeSet whose compare() uses name only (not consistent with equals on regNo)
PREDICTION: two DIFFERENT students with the same name 'Ravi' -> TreeSet keeps only 1 (size 1),
            although equals() says they are different.
OBSERVED  : r1.equals(r2) = false, TreeSet size = 1, contents = [192311020 Ravi]

VERIFICATION of the corrected class (equals/hashCode contract checks)
            all contract checks passed = true

=== De-duplication of n scans (half are repeat scans) : time in ms, average of 5 runs ===
     scans     unique      HashSet      TreeSet      HashMap
     10000       5000         1.38         3.07         3.09
    100000      50000        12.28        44.72        13.41
   1000000     500000       183.42       341.37        91.77
```

Figure 3.2 – Console output of HashContractExperiments.java

Q4 – Choosing a Map for a Pharmacy Inventory

Problem context: stock is looked up by medicine code hundreds of times a day, and every evening two reports are printed: all medicines in **code order**, and all medicines in the **order they were added**.

4.1 Design reasoning before implementation

	HashMap	TreeMap	LinkedHashMap
Inside	array of buckets (hash table)	red-black tree sorted by key	hash table + doubly linked list through the entries
put / get	O(1) average	O(log n)	O(1) average (a little more work than HashMap to maintain the links)
Iteration	O(n + capacity), no useful order	O(n), in key order	O(n), in insertion order
Report in code order	needs extra work: copy keys and sort, O(n log n)	natural: iterate, O(n)	needs extra work: copy keys and sort, O(n log n)
Report in added order	needs extra work: store an "added number" in each Medicine and sort by it, O(n log n)	needs the same extra work, O(n log n)	natural: iterate, O(n)

Table 4.1 – What I expected before implementing

So no single map gives both reports for free: TreeMap gives code order, LinkedHashMap gives added order, and HashMap gives neither. This is the main design tension of this question.

4.2 Implementation and tests

PharmacyInventory.java stores Medicine objects (code, name, stock, addedOrder) in each map. To make the two orders different, the codes (MED0000001 …) are inserted in **random order**. It was tested with three meaningfully different sizes: **1,000** (a small clinic), **20,000** (a large hospital) and **500,000** (a stress test). For each map I measured the time to put all medicines, 100,000 random lookups, a full iteration, and both reports, and I checked every report against the correct answer (a sorted array of the codes and the original insertion order). I also included Hashtable and a sorted array (for the alternative concept).

A small demonstration with six medicines confirms the order behaviour:

```
Added order       : [M305, M102, M550, M101, M420, M210]
HashMap keys      : [M550, M102, M101, M420, M210, M305]   (no guaranteed order)
TreeMap keys      : [M101, M102, M210, M305, M420, M550]   (code order)
LinkedHashMap keys: [M305, M102, M550, M101, M420, M210]   (added order)
```

4.3 Measured results (ms, average of 5 runs; 100,000 lookups)

N	Structure	put all	100k lookups	ns / lookup	iterate	code-order report	added-order report
1,000	HashMap	0.07	4.42	44.2	0.04	0.23	2.42
1,000	TreeMap	0.24	32.27	322.7	0.06	0.05	1.44
1,000	LinkedHashMap	0.07	3.72	37.2	0.02	0.48	0.01
1,000	Hashtable	1.01	4.31	43.1	0.03	0.99	1.34
1,000	SortedArray+binarySearch	2.14	30.49	304.9	0.00	0.02	3.02
20,000	HashMap	3.05	3.67	36.7	1.19	16.77	11.02
20,000	TreeMap	20.11	71.71	717.1	4.07	3.15	8.87
20,000	LinkedHashMap	2.42	6.97	69.7	0.23	22.42	1.05
20,000	Hashtable	5.19	8.51	85.1	0.99	4.72	11.44

N	Structure	put all	100k lookups	ns / lookup	iterate	code-order report	added-order report
20,000	SortedArray+binarySearch	25.10	78.69	786.9	0.03	0.11	10.17
500,000	HashMap	74.75	10.09	100.9	38.60	219.94	205.59
500,000	TreeMap	777.88	172.82	1728.2	72.96	68.50	267.65
500,000	LinkedHashMap	58.68	10.26	102.6	17.25	511.08	32.13
500,000	Hashtable	43.10	13.56	135.6	49.73	268.25	203.67
500,000	SortedArray+binarySearch	632.91	199.92	1999.2	4.08	15.95	201.83

Table 4.2 – Q4 results (from Q4/q4_map_results.csv). In every row the output order matched the requirement (code_order_correct and added_order_correct are true for all structures)

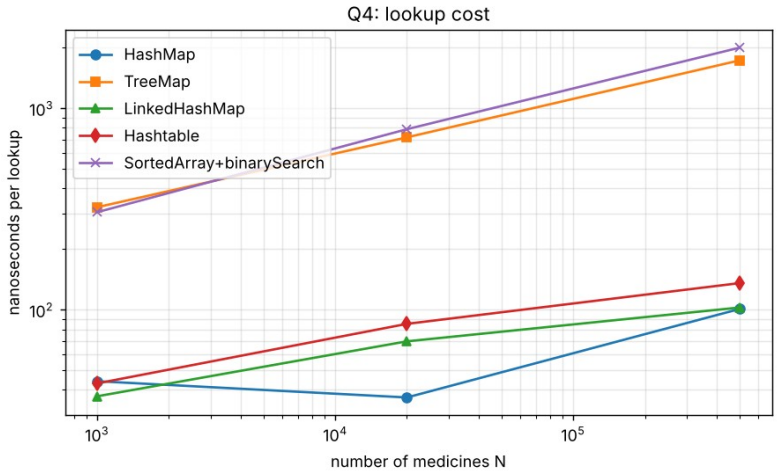


Figure 4.1 – Cost of one lookup

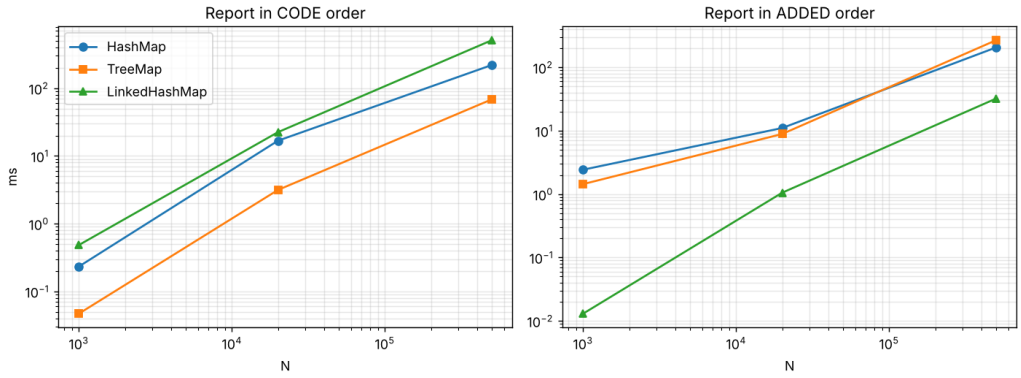


Figure 4.2 – Time of the two evening reports

Observations

- **Lookup:** at N = 500,000 HashMap and LinkedHashMap need about 100.9 ns per lookup and TreeMap 1728.2 ns (about 17 times more), as O(1) against O(log n) predicts ($\log_2$ 500,000 ≈ 19 String comparisons). The cost of the hash maps also rose from about 40 ns (N = 1,000) to 100 ns (N = 500,000) although O(1) is "constant": the table no longer fits in the CPU cache.
- **Reports:** the "natural" map is always the cheapest for its report: TreeMap produced the code-order report in 68.50 ms against 219.94 ms for HashMap (which must sort), and LinkedHashMap produced the added-order report in 32.13 ms against about 200–270 ms for the other two (which must sort by the addedOrder field). This confirms the "extra work" I predicted: an O(n log n) sort.

- **Order check:** all five structures produced both reports in the correct order, because the extra work (sorting) was done; so the choice is about cost, not about being able to produce the report at all.

4.4 Scalability inquiry

Acceptable response time. For a pharmacist standing at a counter, anything below about 100 ms is instant, so I define the acceptable limit for **one lookup as 1 ms** (a deliberately strict limit, because the same server will also be doing other work), and for the evening reports **1 second** for the largest hospital size.

Decision: LinkedHashMap<String, Medicine>. Every map meets the limit by a very large margin (even TreeMap at 500,000 medicines needs only 1.7 µs per lookup). The LinkedHashMap has the fastest lookups (102.6 ns at 500,000), gives the added-order report for free, and the code-order report is a single sort of the keys each evening (about 22.4 ms for 20,000 medicines, about half a second at 500,000 – acceptable once a day). TreeMap is a close second (it makes the code report free but makes lookups 17 times slower and the added report expensive); since lookups are the frequent operation, I preferred the structure that makes lookups fastest.

Where the remaining cost comes from. For the hash-based maps the remaining lookup cost is computing/comparing the String key and memory access (cache misses) when the table is large. For TreeMap it is about $\log_2 n$ String comparisons plus pointer chasing. For the evening report the remaining cost is the O(n log n) sort of String keys and the creation of the temporary list.

Why Hashtable is not preferred in new code. Hashtable is a legacy class from Java 1.0: every method is synchronized, so threads queue up on one lock even when they only read; it forbids null keys and values; it has no ordered variant; and it is slower than HashMap in my tests (for example 85 ns against 37 ns per lookup at N = 20,000). If thread safety is needed, ConcurrentHashMap (fine-grained locking) is the modern choice; otherwise HashMap/LinkedHashMap.

4.5 Alternative concept and complexity analysis

Sorted array + Arrays.binarySearch(): keep a Medicine[] sorted by code (sort once with Arrays.sort), and find a medicine with Arrays.binarySearch(array, key, comparator).

	HashMap	TreeMap	Sorted array + binarySearch
Lookup	O(1) average	O(log n)	O(log n) (measured: 305 / 787 / 1,999 ns at 1k / 20k / 500k)
Insertion of one new medicine	O(1) average	O(log n)	O(n): find the place with binary search, then shift elements; building the whole array once is O(n log n)
Code-order report	O(n log n) (sort keys)	O(n)	O(n) (already sorted; measured 16 ms at 500k)
Space	table + one entry per element (about 32–48 bytes each)	one tree node per element (about 40 bytes)	one reference per element (4–8 bytes): the smallest

Table 4.3 – Alternative concept

The sorted array is attractive when the stock list is **mostly read and rarely changed** (small memory, fastest code-order report) but every new medicine costs O(n) shifting, and it cannot give the added-order report without another structure. Because pharmacies add new medicines regularly during the day, I keep my decision of LinkedHashMap.

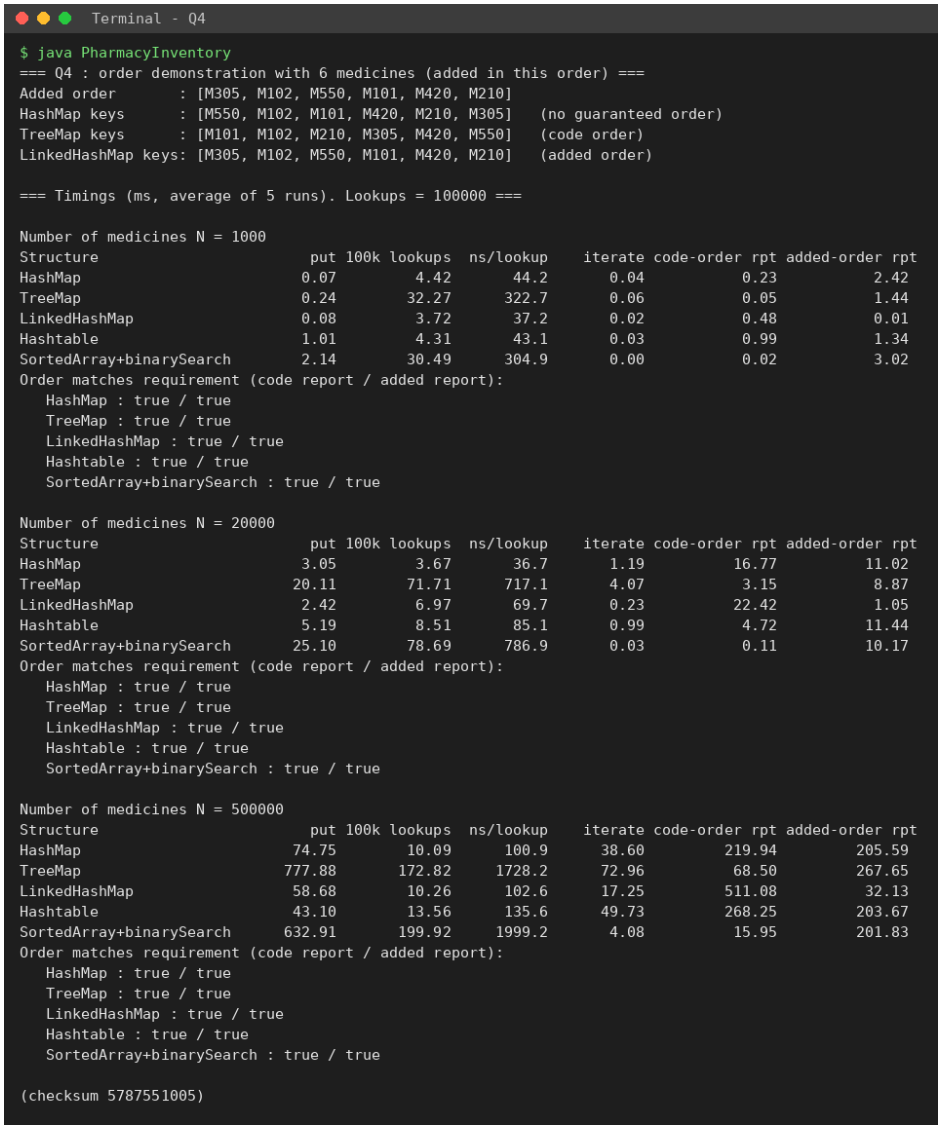


Figure 4.3 – Console output of PharmacyInventory.java

Q5 – Deriving a Generic Ranking Component from a Principle

Principle given: a ranking is correct only if the comparison is *consistent*: if A ranks above B and B above C then A must rank above C, and equal items must compare as equal. **Ranking rule:** points (higher first), then goal difference (higher first), then team name (A–Z). The component must be reusable for players, colleges and quiz teams.

5.1 My design, developed before implementation

Turning the principle into rules for compare(a, b). I translated the principle into four requirements that a comparator must satisfy: (1) **antisymmetry** – sign(compare(a,b)) = −sign(compare(b,a)); (2) **transitivity** – if a ranks above b and b above c then a above c; (3) **equal means equal** – compare(a,b) = 0 means a and b are interchangeable in every comparison with any c; (4) it must never overflow or depend on the order of the input.

Comparable or Comparator?

I decided to use **Comparator objects**, and to keep Comparable only as an optional default. Reasons: a Team has several sensible orders (by rank, by name for a fixture list, by goal difference), and Comparable allows only **one** natural order inside the class; the Leaderboard must be reused for players and colleges, where the "rank order" is different, and I cannot add compareTo to classes I do not own; and with a Comparator I can change the ranking rule (e.g. a new tie-breaker) without touching Team. Comparable is still useful when a type has one obvious order, so I also made a bounded-type board for that case.

Expressing the multi-field comparison

```
Comparator<Team> byRank = Comparator.comparingInt((Team t) -> t.points).reversed()
        .thenComparing(Comparator.comparingInt((Team t) -> t.goalDiff).reversed())
        .thenComparing(t -> t.name);
```

Two details matter. First, **reversed() must be applied to each key's comparator before chaining**; if I wrote comparingInt(points).thenComparing(goalDiff).thenComparing(name).reversed(), the name would also be reversed (Z–A), which is wrong. Second, I use Integer.compare through comparingInt and never write b.points − a.points, because subtraction can overflow (shown in the tests below).

Generic Leaderboard<T> and wildcards

- **Leaderboard<T>** stores a List<T> and a Comparator<? super T>. The wildcard `? super T` is used because the comparator only *consumes* T values: a Comparator<Team> can rank a Leaderboard<CollegeTeam> (shown in the test program). The ranking() method sorts a **copy**, so the stored list is not changed.
- **Bounded type:** `NaturalBoard<T extends Comparable<? super T>>` works for any type that can compare itself, and gives a compile-time error for types that cannot.
- **Where `List<? extends T>` is useful:** an addAll(List<? extends T> more) method can accept a List<CollegeTeam> into a board of Team, because the list only *produces* T values. This follows the rule "producer extends, consumer super".

5.2 Tests

Data Set A (several ties at every level, including two completely identical teams) and **Data Set B** (designed to break faulty comparators) are in DesignTests.java. I also wrote a checker that tests the principle on **every pair and triple** of items (antisymmetry, transitivity, consistent equal items).

```
Ranking with comparator chain : [Eagles(9,6), Falcons(9,6), Falcons(9,6), Bisons(9,4),
Panthers(9,4),
```

```
   Hawks(6,3), Cobras(6,1), Wolves(6,1), Sharks(3,0), Lions(3,-2), Tigers(3,-2)]
Principle check on every pair/triple of Data Set A -> violations = 0
```

Ties are broken correctly at every level: Eagles/Falcons are tied on points and goal difference and are ordered by name; the two identical Falcons are equal; Cobras/Wolves tie on points and goal difference and are ordered A–Z.

Comparator	Result on Data Set B	Violations found by the checker
Correct chain	Alpha (2,000,000,000 pts) first, Bravo (−2,000,000,000 pts) last, correct tie-breaks	0
Faulty 1: points compared by subtraction	Alpha is placed below Bravo (wrong): 2,000,000,000 − (−2,000,000,000) overflows int and becomes negative	12 (e.g. Alpha > Charlie > Bravo, but not Alpha > Bravo – transitivity broken)
Faulty 2: ignores the tie-breakers	Same teams in a different input order give a different ranking (Charlie, Delta, Echo, Zulu vs Zulu, Echo, Delta, Charlie): it is not a function of the data	equal items are not really equal

Table 5.1 – Breaking faulty comparators

I also sorted 2,000 random teams with extreme points using the subtraction comparator. The JDK did not throw an exception in this run (TimSort only sometimes detects a broken comparator with "Comparison method violates its general contract"), but the order was still wrong. So "no exception" is not a proof of correctness – this is why the pair/triple checker is useful.

5.3 Correctness inquiry – why my comparator satisfies the principle

Reasoning, not only program output: the chain compares the keys in the order (points descending, goal difference descending, name ascending) and moves to the next key **only if all earlier keys are equal**. This is the **lexicographic order** on the tuple (−points, −goalDiff, name). (a) Each single key comparison (Integer.compare, String.compareTo) is a total order on its key, and reversing a total order gives a total order. (b) For two teams, find the first key where they differ; the result is decided by that key, and swapping the teams swaps the sign at the **same** key, so **antisymmetry** holds. (c) **Transitivity:** if A ranks above B (first difference at key i) and B above C (first difference at key j), take k = min(i, j). On all keys before k the three teams are identical, and at key k either A is above B (or B is above C) strictly while the other pair is not below, so A is above C at key k; therefore A ranks above C. (d) **Equal items:** compare(A,B) = 0 only when all three keys are equal, and then A and B behave identically against every C. Using Integer.compare means there is no overflow. Therefore the comparator is a consistent total preorder, which is exactly the principle.

5.4 Complete the Code

```
List<T> ranking() {
    // TODO 1 (completed): new sorted copy, items is not modified
    List<T> copy = new ArrayList<>(items);
    copy.sort(order);
    return copy;
}
// in main:
// TODO 2 (completed): the comparator chain shown in section 5.1
Comparator<Team> byRank = Comparator.comparingInt((Team t) -> t.points).reversed()
        .thenComparing(Comparator.comparingInt((Team t) -> t.goalDiff).reversed())
        .thenComparing(t -> t.name);
```

```
Terminal - Q5
$ java LeaderboardDemo
[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]
```

The output matches the expected output exactly. **Why Bisons is ranked above Panthers:** both have 9 points and goal difference 4, so the first two keys are equal and the comparator reaches the third key, the name; "Bisons" comes before "Panthers" alphabetically (B before P), so Bisons ranks higher. Eagles is first because its goal difference (6) is higher than 4 with the same points.

5.5 Alternative concept and complexity analysis

Design	How it works	Sorts	Time	Extra space	Reuse for players / colleges
Comparator chain (my design)	one sort with the chain comparator	1	O(n log n) comparisons, each up to 3 key checks	TimSort temporary array, up to n/2 references = O(n); a few comparator objects	Easy: write a new comparator, Leaderboard<T> unchanged
Comparable<Team>	compareTo contains the same 3 rules; Collections.sort(list)	1	O(n log n)	same O(n) temporary array	Harder: one fixed order per class; every class needs its own compareTo; no second ordering
3 stable sorts	sort by name, then by goal difference, then by points (least important key first)	3	3 × O(n log n) = still O(n log n) but with about 3× the constant	O(n) temporary array each time (reused after each sort)	Fragile: correct only if the order of passes is right and the sort is stable

Table 5.2 – Alternative designs

Measured on n random teams with many ties (average of 5 runs):

n teams	Comparator chain, 1 sort (ms)	Comparable, 1 sort (ms)	3 stable sorts (ms)	Same order?
10,000	7.09	4.61	12.73	true
100,000	70.21	48.10	111.00	true
500,000	533.16	347.40	723.55	true

Table 5.3 – Sorting time (from Q5/q5_sort_results.csv); List.sort and Collections.sort were both used

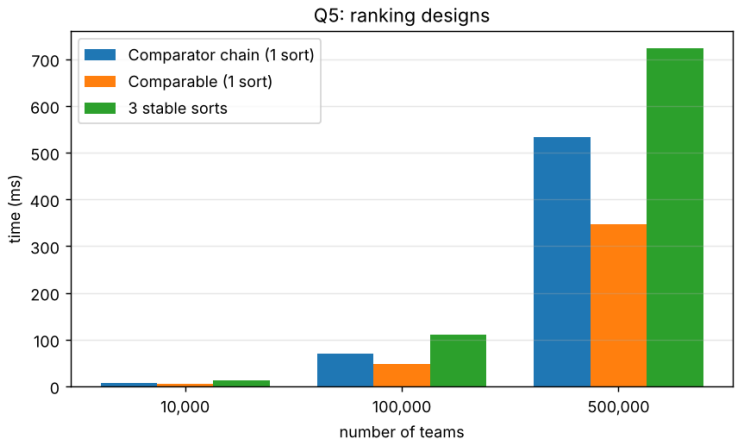


Figure 5.1 – Ranking designs

All three designs gave exactly the same order, so they are equally correct. Comparable was about 1.5 times faster than the comparator chain (it avoids calling several comparator objects), and three sorts took about 1.4–1.8 times as long as one chain sort – less than 3× because TimSort exploits the partly ordered data left by the earlier sort. For **reuse**, the comparator chain is the easiest, which is why I

chose it: the same Leaderboard<T> works for players or quiz teams by passing a different Comparator, and Comparable is the better choice only for a class with one obvious natural order.

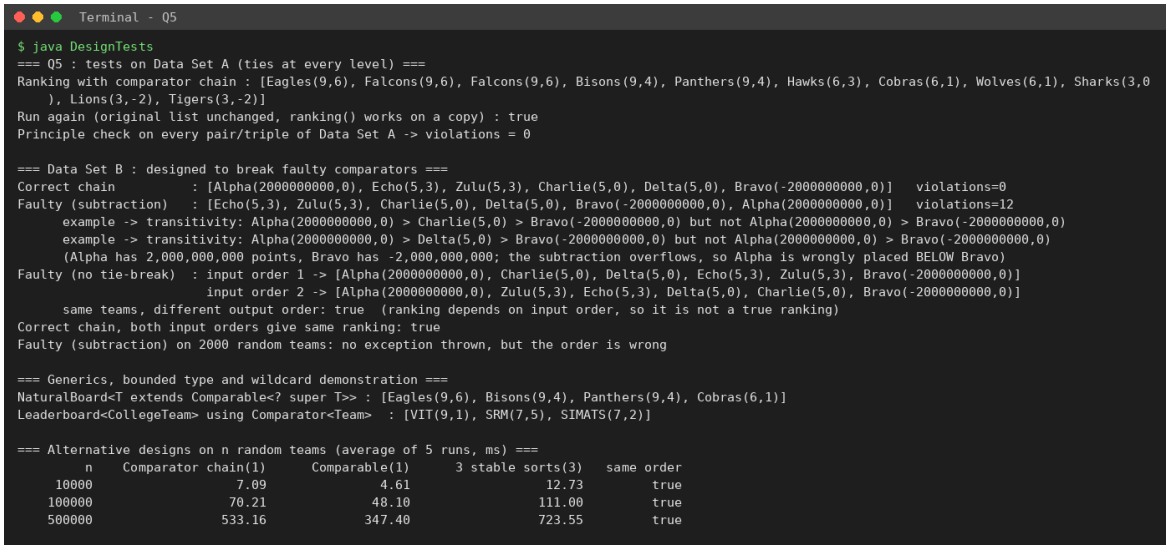


Figure 5.2 – Console output of DesignTests.java

Q6 – Priority, Order and Fairness in a Queue

Problem context: an emergency department gives each patient a severity from 1 (critical) to 5 (minor). The doctor must always see the most severe patient next; among equal severities, the patient who arrived first.

6.1 Design

- **Why a plain Queue is not enough.** A Queue (LinkedList or ArrayDeque) is first-in-first-out: it serves strictly by arrival. In my 8-patient example it would serve Chitra (S5, arrived third) before Devi (S1, arrived fourth), which is dangerous. A Queue has no way to "skip ahead".
- **PriorityQueue with a Comparator.** A PriorityQueue is a binary heap that always keeps the "smallest" element (according to its comparator) at the head. With the comparator "severity number ascending" the most critical patient is always at the head.
- **Preserving arrival order among equal severities.** A heap is **not stable**: two elements that compare equal come out in an unpredictable order. I therefore give every patient an increasing arrival counter (counter++ when admitted) and compare (severity, arrival). Because the counter is unique, two patients are never "equal", so the order is fully determined. My experiment shows the problem: with a severity-only comparator the output order was Esha before Arjun (both S3), although Arjun arrived first.
- **Next patient and display.** next() = queue.poll(), which removes the head. To display the waiting list **in treatment order** I copy the queue (new PriorityQueue<>(queue)) and poll the copy until empty (O(n log n)), or put the elements into a list and sort with the same comparator; the original queue is untouched.

6.2 Correctness check with 8 patients (worked out by hand)

Arrival #	Patient	Severity
0	Arjun	3
1	Bala	2
2	Chitra	5
3	Devi	1
4	Esha	3
5	Farid	2
6	Gita	1
7	Hari	4

By hand: severity 1 first, in arrival order: Devi (3), Gita (6); then severity 2: Bala (1), Farid (5); severity 3: Arjun (0), Esha (4); severity 4: Hari; severity 5: Chitra. Expected order: **Devi, Gita, Bala, Farid, Arjun, Esha, Hari, Chitra**.

```
System.out.println(priorityQueue):
[Devi(S1), Bala(S2), Gita(S1), Arjun(S3), Esha(S3), Chitra(S5), Farid(S2), Hari(S4)]
Treatment order using poll()  : Devi Gita Bala Farid Arjun Esha Hari Chitra
Hand-worked expected order    : Devi Gita Bala Farid Arjun Esha Hari Chitra
Program matches hand calculation : true
```

The program matches the hand calculation. **Why printing the PriorityQueue directly does not show the treatment order:** PriorityQueue.toString() prints the internal array of the binary heap in level order. The heap only guarantees that every parent is not larger than its children, not that the array is sorted; only the first element (the head) is guaranteed to be the next patient. In the printed array, Gita (S1) comes after Bala (S2). Only repeated poll() gives the sorted order.

6.3 Complete the Code

```
// TODO 1 (completed)
private PriorityQueue<Patient> queue = new PriorityQueue<>(
        Comparator.comparingInt((Patient p) -> p.severity)
                  .thenComparingLong(p -> p.arrival));
```

```
Terminal - Q6
$ java Triage
Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)
```

The output equals the expected output. **How the arrival counter keeps equal severities in arrival order:** the counter is increased for each admitted patient, so Meena (counter 1) has a smaller arrival value than Kumar (counter 4). When two patients have the same severity, the comparator's second rule thenComparingLong(arrival) compares those values and the smaller one (the earlier arrival) is placed first. The same rule gives Ravi (counter 0) before John (counter 2) for severity 3.

6.4 Growth investigation

Prediction: add costs O(log n) in the worst case (the new element may move up the heap), but for random data the average is close to O(1) because a new element usually moves up only a level or two. Removing the head (poll) takes O(log n): the last element is moved to the root and sifted down, up to about $2 \cdot \log_2 n$ comparisons. So when n grows by a factor of 1,000 (1,000 → 1,000,000), $\log_2 n$ only doubles (10 → 20), so I expected poll to become only about 2–3 times slower, and add to stay roughly constant.

n patients	add (ns / op)	poll (ns / op)	all adds (ms)	all polls (ms)
1,000	20.7	112.0	0.02	0.11
10,000	28.2	192.8	0.28	1.93
100,000	32.8	246.7	3.28	24.67
1,000,000	37.6	425.0	37.57	425.00

Table 6.1 – PriorityQueue cost (average of 30 runs; 5 runs for n = 1,000,000), from Q6/q6_growth_results.csv

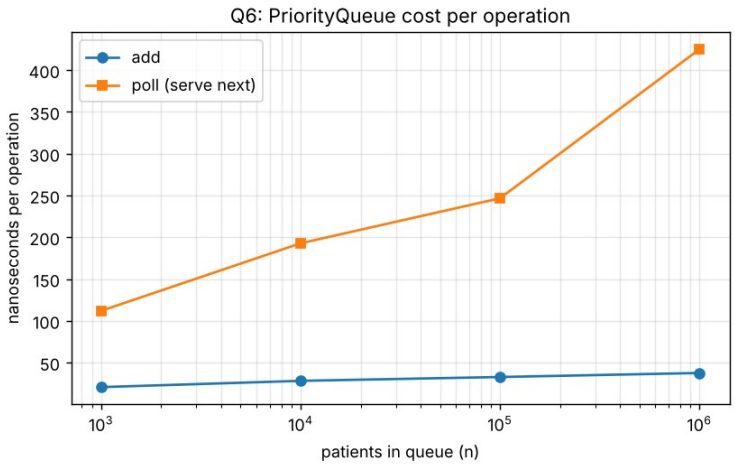


Figure 6.1 – Cost per operation

Observed: add stayed between about 20.7 and 37.6 ns (almost constant), and poll rose from about 112.0 ns to 425.0 ns, i.e. about 3.8 times for 1,000 times more patients. This agrees with the

prediction (logarithmic growth, plus slower memory access when the heap is big). A linear structure would have become 1,000 times slower.

6.5 Starvation and a fair rule

Why a severity-5 patient may wait indefinitely. The comparator compares severity first. As long as at least one patient with a smaller severity number is in the queue, the severity-5 patient can never reach the head. If critical patients keep arriving at least as fast as the doctor can treat them, the head is always a more severe patient. The arrival counter does not help, because it is used only when the severity is equal.

Fair rule – aging. Every 50 time units of waiting lowers a patient's *effective severity* by one level (never below 1): effective = max(1, severity − waited / 50). The queue is ordered by (effective severity, arrival). A severity-5 patient is then treated at the latest after 4 × 50 = 200 time units, and a severity-1 patient is never delayed by more than the effects of other patients who have also waited long.

```
Plain PriorityQueue : severity-5 patient served at time NEVER (still waiting after
2000 time units)
Fair rule (aging, limit 50) : severity-5 patient served at time 200
```

In this worst-case simulation (one new critical patient arrives and one patient is served at every time unit) the plain queue never serves the severity-5 patient, while the fair rule serves him at exactly time 200, as designed. (Note: the fair rule changes priorities over time, so a normal PriorityQueue cannot keep them; my simple implementation rebuilds the queue before every serve. Its cost is analysed in Q7.)

6.6 Deployment decision for 5,000 patients a day

```
Plain PriorityQueue : admit + serve 5,000 patients = 8.98 ms in total (1.797
microseconds per patient)
Fair (rebuild) rule : admit + serve 5,000 patients = 286.48 ms in total (57.296
microseconds per patient)
```

5,000 patients per day is about one patient every 17 seconds. Even in the worst case where all 5,000 are waiting together, the plain priority queue needs about 9 ms for the whole day, and my growth test handled 1,000,000 patients in well under a second. **So yes, the design can serve a city hospital with 5,000 patients a day** – the data structure is thousands of times faster than the doctors; the real bottleneck is clinical capacity. The fair version with rebuilding also works for 5,000 (0.3 s for the whole day) but its cost grows with n^2 (see Q7), so it is not suitable for much larger queues. Limits of my evidence: single-threaded test; a real system also needs thread safety (e.g. PriorityBlockingQueue) and persistence.

6.7 Alternative concept and complexity analysis

Design	Admit	Serve next	Space	Remark
PriorityQueue (my design)	O(log n)	O(log n)	O(n) – one array	General: works for any number of priority levels
Sorted LinkedList (insert in position)	O(n): walk to the right position (first patient with larger severity)	O(1): pollFirst()	O(n) – one node object (about 24 bytes) per patient	Simple to read but slow to admit
Array of 5 FIFO queues (one per severity)	O(1): addLast to queue [severity − 1]	O(1): check the 5 queues in order (at most 5 steps, a constant)	O(n) – 5 ArrayDeques	Fast, but only for a small fixed number of levels

Table 6.2 – Alternatives

n	Design	admit all (ms)	serve all (ms)
1,000	PriorityQueue	0.061	0.202
1,000	Sorted LinkedList	0.663	0.008
1,000	Five FIFO queues	0.038	0.011
5,000	PriorityQueue	1.027	0.995
5,000	Sorted LinkedList	63.728	1.317
5,000	Five FIFO queues	0.559	0.018
20,000	PriorityQueue	0.786	5.385
20,000	Sorted LinkedList	656.180	0.198
20,000	Five FIFO queues	0.119	0.077

Table 6.3 – Measured (from Q6/q6_alternatives_results.csv; average of 5 runs after warm-up)

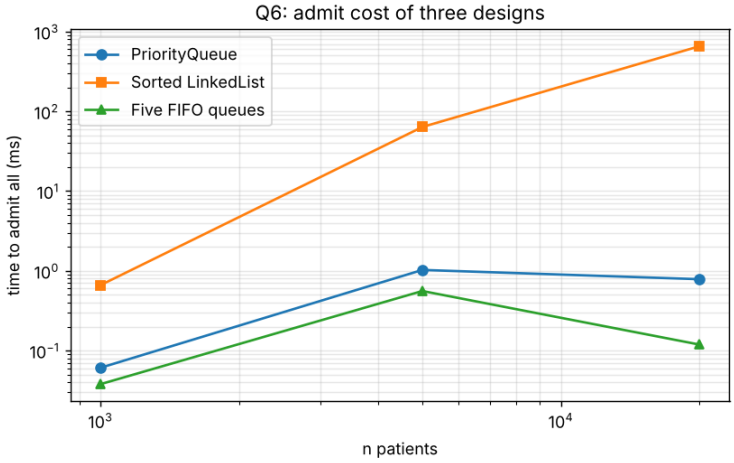


Figure 6.2 – Admit cost of the three designs

The measurements match the predictions: the sorted LinkedList admit time grows roughly with n^2 for n admissions (0.7 ms → 64 ms → 656 ms as n grows 1,000 → 5,000 → 20,000), while the PriorityQueue and the five FIFO queues stay small. For serving, the sorted LinkedList is O(1), as predicted. The five queues are the fastest here, but they depend on having only five severity levels, whereas the PriorityQueue still works if the priority is a score with thousands of values.

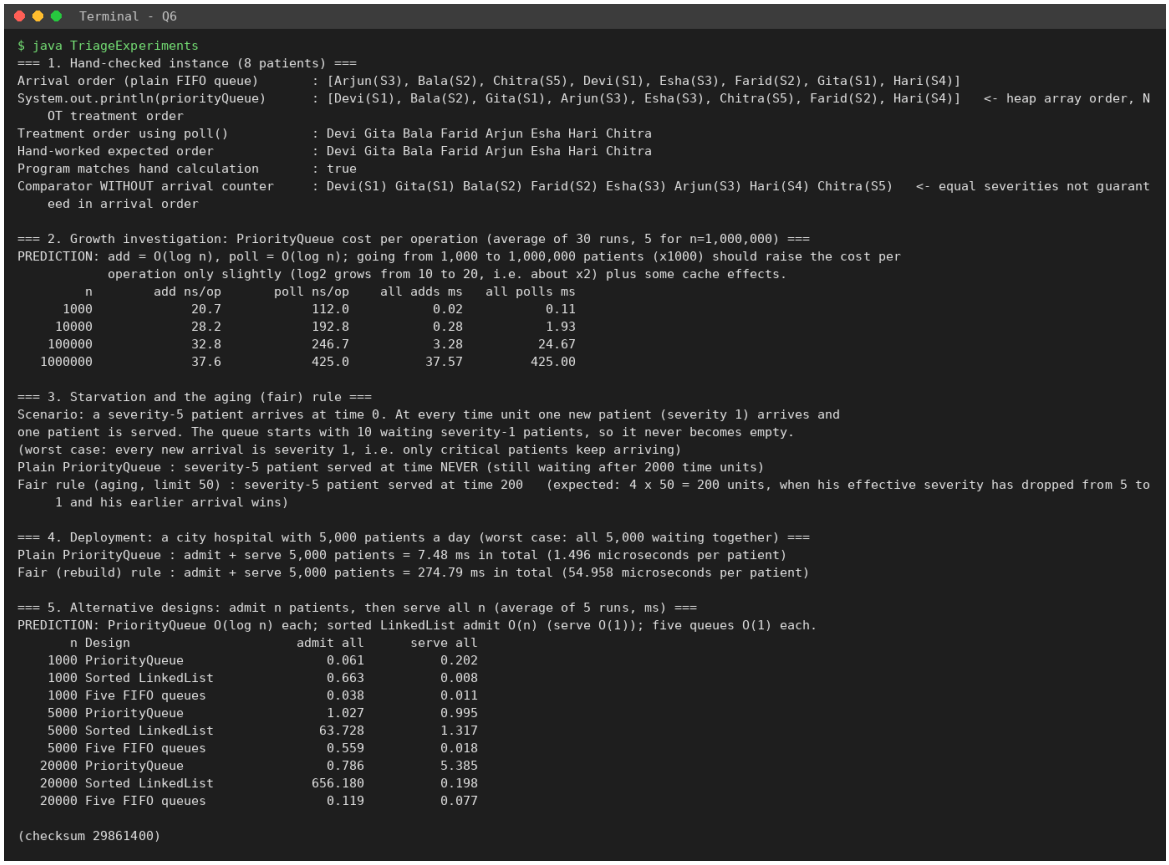


Figure 6.3 – Console output of TriageExperiments.java

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate

Solution selected: the fair emergency-triage queue from Q6 (aging rule). From the evidence in Q6 it is the solution whose behaviour changes most with the data size: it costs 57 µs per patient for 5,000 patients (against 1.8 µs for the plain queue) and, as the experiment below shows, its total cost grows with the *square* of the queue length.

A. Diagnose the bottleneck (2 marks)

Original solution. Because the effective severity of every patient changes as time passes, a normal PriorityQueue (whose order is fixed when the elements are inserted) cannot be used directly. My Q6 solution therefore **rebuilds the PriorityQueue before every serve**: it creates a new PriorityQueue with a comparator using the current time, adds all waiting patients into it, and polls one.

```
Patient serve(long now) {
    Comparator<Patient> cmp = comparingInt(p -> effective(p, now,
waitLimit)).thenComparingLong(p -> p.seq);
    PriorityQueue<Patient> fresh = new PriorityQueue<>(cmp);   // new queue + new
array
    fresh.addAll(queue);                                       // re-examines EVERY
waiting patient
    queue = fresh;
    return queue.poll();
}
```

Question	Diagnosis from my evidence
Where is most of the time / memory spent?	In fresh.addAll(queue): every serve re-inserts all waiting patients into a new heap. For n = 20,000 the program re-examined 200,010,000 patients in total, took 4.6 s and allocated 2.76 GB of memory (almost all of it temporary queues and arrays that become garbage).
Which operation grows significantly with the data?	serve(). Its cost is proportional to the queue length, so serving n patients costs 1 + 2 + … + n = n(n+1)/2 = O(n²). The examined-patient count is exactly n(n+1)/2 (for n = 20,000: 200,010,000), and time grew about 3.2 times when n doubled from 5,000 to 10,000 and about 3.5 times from 10,000 to 20,000 (close to the factor 4 expected for n²).
Is work unnecessarily repeated?	Yes. When one time unit passes, the relative order of most patients does not change, yet the whole queue is re-sorted and rebuilt from scratch before every single serve; 5,000 PriorityQueue objects were created for 5,000 patients.
Which characteristic of the chosen class causes the limitation?	A PriorityQueue (a heap) assumes that an element's priority does not change while it is inside – the same kind of problem as changing a field used in hashCode() in Q3. The heap cannot be "updated" cheaply for all elements, so my only option with that class was to rebuild it.

Table 7.1 – Diagnosis

B. Design objective (2 marks)

The redesign must: (1) serve the next patient in time that **does not depend on the queue length n** (a constant number of steps per serve), and so reduce the total examined patients for n = 20,000 from about 2×10^8 to at most 5 per serve (≤ 100,000 in total); (2) create **no new queue objects per serve** (target: 0 temporary queues, memory for the whole run below 1 MB instead of 2.7 GB); (3) preserve **correctness**: the order of service must be *identical* to the original for the same input, including all ties; (4) be at least 100 times faster for n = 20,000.

C. Design of the revised solution (4 marks)

What I changed. I replaced the single PriorityQueue by **five FIFO queues, one per severity level** (ArrayDeque). A patient is added at the tail of the queue for his severity. To serve, I do **not** look at all patients: I compare only the **head** (oldest patient) of each of the five queues using the effective severity at the current time, and remove the best one.

Which property I exploit. Inside one severity queue, patients are in arrival order, and arrival times never decrease. A patient who arrived earlier has waited longer, so his effective severity is **less than or equal to** that of anyone behind him in the same queue. Therefore the best patient of each level is always its head, and the best patient overall is one of only five heads. Ties are resolved by arrival number (seq), exactly as in the original, so the order is identical.

Operations avoided or reduced. Re-adding all waiting patients, creating a new heap and array, and comparing every patient with every other are avoided. Admit is O(1); serve is O(K) with K = 5 (a constant).

Extra memory / preprocessing. None: five empty deques are created once. Memory use per patient is about the same as before (one array slot).

New trade-offs. (1) The number of levels K is fixed (here 5): the cost per serve is O(K), so with thousands of levels a heap of heads would be needed. (2) The idea works only because the aging rule depends only on severity and arrival time; a rule that also depended on, say, the patient's age would break the "head is the best" property. (3) It needs the clock to be non-decreasing, which is natural for arrival times.

Design representation (pseudocode and class sketch).

```
class FiveQueueTriage
    fields : ArrayDeque<Patient>[] q = new ArrayDeque[5]     // q[0] = severity 1 ...
q[4] = severity 5
             long waitLimit

    admit(p)           :  q[p.severity - 1].addLast(p)                       // O(1)

    serve(now)         :  best = null
                          for level = 0 .. 4                                 // 5
steps, constant
                              head = q[level].peekFirst()                    // oldest
of that level
                              if head != null and
                                 (best == null or better(head, best, now))   //
effective severity, then seq
                                  best = head
                          return best == null ? null : q[best.severity -
1].pollFirst()

    better(a, b, now)  :  ea = max(1, a.severity - (now - a.time) / waitLimit)
                          eb = max(1, b.severity - (now - b.time) / waitLimit)
                          return ea < eb  or  (ea == eb and a.seq < b.seq)
```

Class	Fields	Methods
Patient	name, severity, seq (arrival number), time (arrival time)	toString()
RebuildTriage (original)	PriorityQueue queue, waitLimit, counters	admit(p), serve(now)
FiveQueueTriage (redesign)	ArrayDeque<Patient>[5] q, waitLimit, counter	admit(p), serve(now)
TriageFair (test driver)	–	drain…(), main(): equivalence tests, timing, counting

Table 7.2 – Classes

D. Prediction before implementing (1 mark)

Written before running the experiment: (1) the redesign examines at most 5 queue heads per serve, so for n patients about 5n in total (n = 20,000 → about 100,000), while the original examines n(n+1)/2 (200,010,000); the operation count should therefore improve by a factor that **grows with n**

(about 2,000× at n = 20,000). (2) Time should improve by at least 100× at n = 20,000, and the redesign's time should grow roughly linearly (not quadratically) with n. (3) Memory allocated should drop from gigabytes to well under 1 MB. (4) The order of service must be identical in all tests. The measure where I expect the clearest improvement is the **number of patients examined**, because timings of a few milliseconds are noisy.

E. Implement and validate (2 marks)

Same test conditions for both designs: n patients all waiting (100 arrivals per time unit, random severities with a fixed seed), wait limit 500 time units, and the doctors serve 10 patients per time unit until the queue is empty; average of 5 runs (3 for n ≥ 10,000) after warm-up. I also ran **300 random mixed sequences** (admit and serve steps in random order, small wait limits so that aging happens often); 40,237 service decisions were compared and there were **0 mismatches**, so the redesign gives exactly the same order as the original.

n	Design	Time (ms)	Patients / heads examined	PQs created	Allocated (MB)	Same order
1,000	Original (rebuild PQ)	11.55	500,500	1,000	7.0	true
1,000	Redesign (5 queues)	0.13	3,043	0	0.0	true
2,500	Original (rebuild PQ)	74.52	3,126,250	2,500	43.1	true
2,500	Redesign (5 queues)	0.95	7,528	0	0.1	true
5,000	Original (rebuild PQ)	406.11	12,502,500	5,000	177.6	true
5,000	Redesign (5 queues)	2.29	14,892	0	0.1	true
10,000	Original (rebuild PQ)	1302.82	50,005,000	10,000	712.6	true
10,000	Redesign (5 queues)	1.68	29,801	0	0.3	true
20,000	Original (rebuild PQ)	4622.79	200,010,000	20,000	2762.7	true
20,000	Redesign (5 queues)	0.92	64,430	0	0.5	true

Table 7.3 – Original vs redesign (from Q7/q7_results.csv)

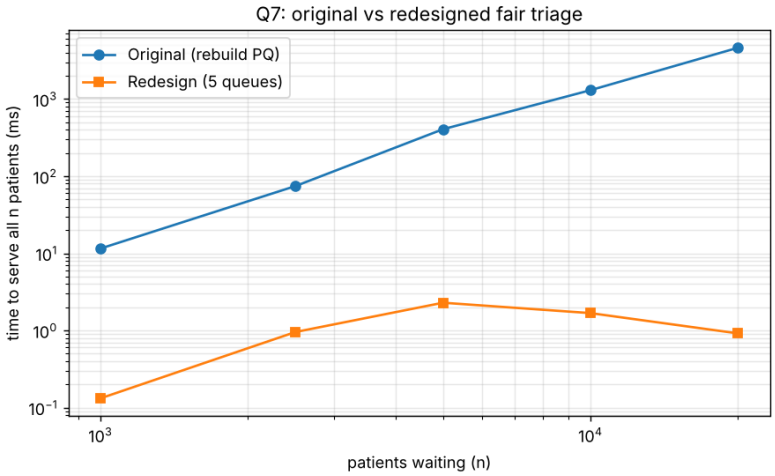


Figure 7.1 – Time to serve all n patients (logarithmic axes)

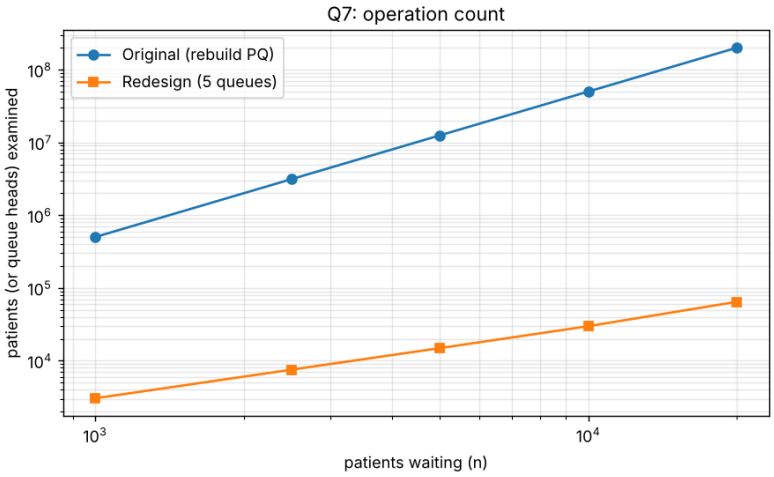


Figure 7.2 – Operation count: $n^2/2$ against about $5n$

The redesign alone on larger queues (the original would take minutes): n = 100,000 took 11.61 ms and n = 1,000,000 took 108.16 ms, i.e. about 108.2 ns per serve in both cases – a constant cost per serve, so the total time grows linearly. The heads examined were 4,999,983 for 1,000,000 patients (almost exactly 5 per serve).

F. Defend the final design (2 marks)

- **Was the prediction supported?** Yes. The examined count fell from 200,010,000 to 64,430 at n = 20,000 (3,104 times fewer; at most 5 heads per serve), no priority queues were created, and the order was identical in all comparisons.
- **Magnitude of improvement:** at n = 20,000 the time fell from 4,623 ms to 0.92 ms (more than 1,000 times faster; the exact ratio depends on timer noise at about 1 ms) and the allocated memory from 2762.7 MB to 0.5 MB. The objective (≥ 100× faster, no temporary queues, identical order) was achieved. The redesign's timings for n between 1,000 and 20,000 are all of the order of a millisecond and are not perfectly monotonic (noise from the JIT compiler and the garbage collector); the operation counts and the 100,000 / 1,000,000 runs show the trend clearly.
- **New costs and trade-offs:** the number of severity levels is fixed (K = 5) and the structure relies on the monotone-time property of the aging rule; it is a special-purpose design, less general than a PriorityQueue.
- **When the original may still be preferable:** if the system has no aging (plain priority), the Q6 PriorityQueue is simpler, has O(log n) operations and supports any number of priority values. For small queues (a few dozen patients) the rebuild cost is negligible and the simple code is easier to read. And if the priority rule was something that cannot be split into FIFO levels (for example a continuous score), the five-queue idea would not apply.

Final engineering conclusion: the limitation came from the characteristics of PriorityQueue (fixed priorities) combined with a time-dependent rule, not from a lack of speed of the class itself. By understanding the problem and using the structure of the data (FIFO inside each level, monotone time) I reduced the work per serve from O(n) to O(1) without changing the result.

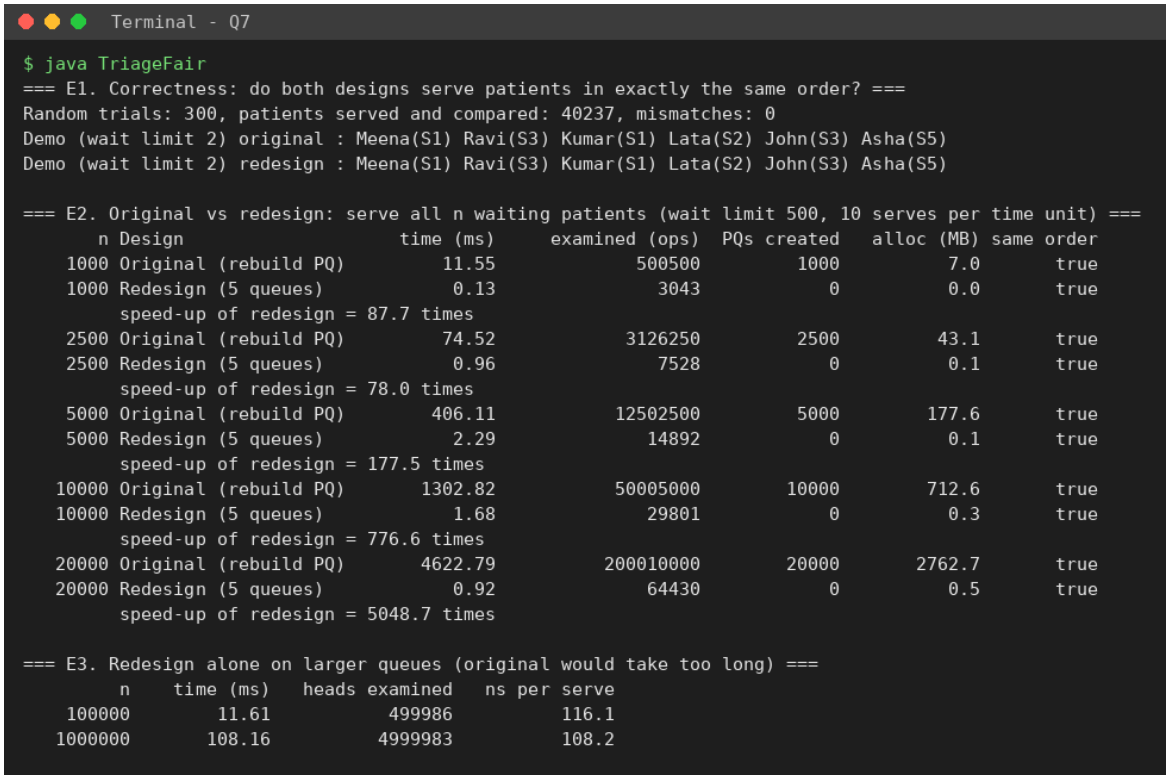


Figure 7.3 – Console output of TriageFair.java

Appendix – Repository, How to Run and Raw Data

A. Repository structure

```
CSA09-<RegisterNumber>-Assignment2/
  README.md
  Q1/  TicketBuilder.java, output.txt, q1_workloads.csv, q1_scaling.csv,
q1_threads.csv, screenshot_TicketBuilder.png
  Q2/  PlaylistExperiments.java, output.txt, q2_list_results.csv,
q2_upnext_results.csv, q2_array_results.csv, screenshot_...png
  Q3/  AttendanceBefore.java, Attendance.java, HashContractExperiments.java, outputs,
q3_dedup_results.csv, screenshots
  Q4/  PharmacyInventory.java, output.txt, q4_map_results.csv,
screenshot_PharmacyInventory.png
  Q5/  LeaderboardDemo.java, DesignTests.java, outputs, q5_sort_results.csv,
screenshots
  Q6/  Triage.java, TriageExperiments.java, outputs, q6_growth_results.csv,
q6_alternatives_results.csv, screenshots
  Q7/  TriageFair.java, output.txt, q7_results.csv, q7_redesign_large.csv,
screenshot_TriageFair.png
  graphs/   (all figures used in this report)
  scripts/  make_graphs.py, make_screenshots.py
  report/   this report
```

B. How to compile and run

Open a terminal in the question folder, for example:

```
cd Q1
javac TicketBuilder.java
java TicketBuilder

cd ../Q3
javac *.java
java AttendanceBefore        (before TODO 1: prints 3)
java Attendance              (after TODO 1: prints 2)
java HashContractExperiments

cd ../Q5
javac *.java                 (LeaderboardDemo.java and DesignTests.java are compiled
together)
java LeaderboardDemo
java DesignTests

cd ../Q6
javac *.java
java Triage
java TriageExperiments
```

C. Raw experimental data files

File	Content
Q1/q1_workloads.csv, q1_scaling.csv, q1_threads.csv	Time, allocation, estimated objects for the 3 workloads; scaling with n; 20 thread trials
Q2/q2_list_results.csv, q2_upnext_results.csv, q2_array_results.csv	Time per operation for the 3 structures at 3 sizes; Up Next queue; plain array

File	Content
Q3/q3_dedup_results.csv	HashSet, TreeSet and HashMap de-duplication time
Q4/q4_map_results.csv	put, lookup, iteration and both reports for 5 structures at 3 sizes, with order-check results
Q5/q5_sort_results.csv	Comparator chain vs Comparable vs 3 stable sorts
Q6/q6_growth_results.csv, q6_alternatives_results.csv	PriorityQueue growth; sorted list and five-queue alternatives
Q7/q7_results.csv, q7_redesign_large.csv	Original vs redesign; redesign at 100,000 and 1,000,000 patients

Every table and graph in this report was produced from these files. If the programs are run again the numbers will change a little (different hardware, JIT and garbage-collection timing), but the trends and the operation counts stay the same.